

## Semana 10 — Quick Sort y Veredicto StreamMX

### COMPRENDE — Quick Sort: paradigma de partición

#### StreamMX — Semana 10: El Veredicto Final

| Distribución              | Características                          | Riesgo para el algoritmo                                                               |
|---------------------------|------------------------------------------|----------------------------------------------------------------------------------------|
| <b>A: Casi ordenados</b>  | Cambios mínimos respecto al día anterior | Algoritmos adaptativos pueden brillar; Merge Sort no aprovecha tanto                   |
| <b>B: Aleatorios</b>      | Datos frescos, sin patrón                | Se mide el rendimiento puro $O(n \log n)$                                              |
| <b>C: Empates masivos</b> | Millones de registros con puntaje igual  | Solo algoritmos <b>estables</b> preservan orden cronológico; inestables rompen ranking |

#### Estrategia de Prueba

- Implementar ambos candidatos:**
  - Merge Sort (baseline).
  - Algoritmo in-place  $O(n \log n)$  (ej. Heap Sort o introsort, pero ojo con estabilidad).
- Dataset sintético + real:**
  - Generar arrays con las tres distribuciones.
  - Para C, incluir campos (fecha, puntaje) y verificar orden cronológico tras el sort.
- Métricas a recolectar:**
  - Tiempo de ejecución (ms).
  - Memoria máxima usada (MB).
  - Estabilidad (booleano: ¿se preserva orden relativo?).
- Veredicto automático:**
  - Script que compare resultados y emita tabla final.
  - Si el algoritmo in-place falla en estabilidad → se descarta.
  - Si cumple estabilidad y reduce RAM → candidato fuerte.

#### Reflexión técnica

- Merge Sort = seguro en estabilidad, pero caro en RAM.
- Heap Sort = in-place, pero inestable (rompe Distribución C).
- Introsort (Quick + Heap fallback) = rápido e in-place, pero tampoco estable.
- **Conclusión preliminar:** si la estabilidad es requisito absoluto, Merge Sort sigue siendo el estándar. El “nuevo algoritmo” tendría que ser una variante in-place estable (muy raro en práctica).

| Algoritmo                               | Distribución       | Tiempo (ms) | Memoria (MB) | Estabilidad (✓/X) | Observaciones |
|-----------------------------------------|--------------------|-------------|--------------|-------------------|---------------|
| <b>Merge Sort</b>                       | A: Casi ordenados  | ...         | ...          | ✓                 | ...           |
|                                         | B: Aleatorios      | ...         | ...          | ✓                 | ...           |
|                                         | C: Empates masivos | ...         | ...          | ✓                 | ...           |
| <b>Nuevo Algoritmo (Heap/IntroSort)</b> | A: Casi ordenados  | ...         | ...          | X/✓               | ...           |
|                                         | B: Aleatorios      | ...         | ...          | X/✓               | ...           |
|                                         | C: Empates masivos | ...         | ...          | X/✓               | ...           |

### Reto IA — Comprende: Cuestionamiento socrático sobre Quick Sort

#### Pregunta 1 — Pivote en posición final

##### Turno 1:

- ¿Qué ocurre con los elementos menores y mayores al pivote durante la partición?

##### Turno 2 (si el equipo responde):

- ¿Cómo sabes que, al terminar, ningún elemento “fuera de lugar” queda a la izquierda o derecha del pivote?

##### Turno 3 (profundización):

- ¿Qué propiedad del arreglo asegura que el pivote ya no necesita moverse más?

#### Pregunta 2 — Invariante con todos iguales

##### Turno 1:

- ¿Qué significa “menor o igual al pivote” si todos los valores son idénticos?

##### Turno 2 (si el equipo responde):

- ¿Cómo se comporta el índice que separa los menores del resto en ese caso?

##### Turno 3 (profundización):

- ¿Qué efecto tendría esto en la eficiencia del algoritmo?

### Pregunta 3 — Datos casi ordenados, pivote último

#### Turno 1:

- Si el arreglo ya está casi ordenado, ¿qué posición ocupa el último elemento respecto a los demás?

#### Turno 2 (si el equipo responde):

- ¿Qué tamaño tendrán las dos sublistas después de la partición?

#### Turno 3 (profundización):

- ¿Cómo afectaría eso a la profundidad de la recursión y al tiempo total?

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
// Función de partición Lomuto (INCOMPLETA — el equipo debe llenarla)
```

```
int lomutoPartition(vector<int>& arr, int low, int high) {
```

```
    int pivot = arr[high]; // pivote = último elemento
```

```
    int i = low - 1;
```

```
    // IA-SOCRATICO: ¿Qué ocurre con los elementos menores y mayores al pivote?
```

```
    // IA-SOCRATICO: ¿Cómo sabes que, al terminar, ningún elemento queda fuera de lugar?
```

```
    // IA-SOCRATICO: ¿Qué propiedad del arreglo asegura que el pivote ya no necesita moverse?
```

```
    for (int j = low; j < high; j++) {
```

```
        // IA-SOCRATICO: ¿Qué significa "menor o igual al pivote"
```

```
        // si todos los valores son idénticos?
```

```
        // IA-SOCRATICO: ¿Cómo se comporta el índice i en ese caso?
```

```
        // IA-SOCRATICO: ¿Qué efecto tendría esto en la eficiencia?
```

```
    }
```

```
    // IA-SOCRATICO: Si el arreglo ya está casi ordenado,
```

```
    // ¿qué posición ocupa el último elemento respecto a los demás?
```

```
    // IA-SOCRATICO: ¿Qué tamaño tendrán las dos sublistas?
```

```
// IA-SOCRATICO: ¿Cómo afectaría eso a la profundidad de la recursión?

// Aquí el equipo debe completar el intercambio final

return i + 1;
}

void quickSort(vector<int>& arr, int low, int high) {
    if (low < high) {
```

APLICA — Implementar, medir y confrontar

Propósito de cada variable

- **pivote:** es el valor de referencia (último elemento) contra el cual se comparan los demás.
- **i:** marca la frontera entre los elementos “menores o iguales al pivote” y los que aún no se han clasificado.
- **j:** recorre el arreglo desde low hasta high-1, comparando cada elemento con el pivote.

2. Pregunta socrática sobre el invariante

- Si en cada paso garantizamos que todo lo que está a la izquierda de i es menor o igual al pivote, ¿qué significa entonces que i avance cuando encontramos un elemento menor o igual?

*(El equipo debe razonar que el invariante asegura que la zona izquierda siempre está bien clasificada respecto al pivote.)*

3. Línea específica que suele causar confusión

Cuando el equipo ya entiende el invariante, la línea que genera dudas es el intercambio final del pivote:

```
cpp
swap(arr[i + 1], arr[high]);
```

Esa línea coloca al pivote en su posición definitiva, justo después de los menores o iguales.

Registro de reflexión

Al terminar, cada integrante debe anotar:

```
// IA-REFLEXION-A: La IA nos aclaró el rol de i, j y pivote.
// IA-REFLEXION-A: Entendimos que el invariante mantiene orden parcial.
// IA-REFLEXION-A: Implementamos el intercambio final sin ayuda.
```

| Algoritmo                  | Distribución | Tiempo (ms) | Memoria (MB) | Observaciones |
|----------------------------|--------------|-------------|--------------|---------------|
| <b>Merge Sort</b>          | Aleatorio    | ...         | ...          | ...           |
|                            | Ordenado     | ...         | ...          | ...           |
|                            | Inverso      | ...         | ...          | ...           |
|                            | Duplicados   | ...         | ...          | ...           |
| <b>Quick Sort (Lomuto)</b> | Aleatorio    | ...         | ...          | ...           |
|                            | Ordenado     | ...         | ...          | ...           |
|                            | Inverso      | ...         | ...          | ...           |
|                            | Duplicados   | ...         | ...          | ...           |

#### Guía de llenado

- **Tiempo (ms):** promedio de varias corridas (ej. 5–10).
- **Memoria (MB):** pico máximo observado.
- **Observaciones:** anotar ventajas o problemas (ej. “rápido pero inestable en duplicados”).

#### Parte C — Estabilidad

Después de llenar la tabla, el equipo debe correr el experimento de estabilidad:

- Ordenar `datos_streamx` con Merge Sort y Quick Sort.
- Usar `verificar_estabilidad()` en ambos resultados.
- Imprimir las películas con puntaje 7.5 y revisar si el orden cronológico se preserva.

#### Reflexiones del equipo

// REFLEXIONA-1: Qué distribución mostró mayor diferencia entre Merge y Quick.

// REFLEXIONA-2: Qué aprendimos sobre estabilidad en duplicados.

// REFLEXIONA-3: Qué algoritmo recomendaríamos con base en datos reales.

#### Reto 4 — Quick Sort configurable: tres estrategias de pivote

Mediana de tres en arreglos casi ordenados

Imagina un arreglo de 5 elementos ya casi ordenados: [2, 3, 4, 5, 6]

- Si eliges el **último elemento** como pivote (6), la partición queda muy desbalanceada: casi todo a la izquierda, nada a la derecha.
- Si eliges el **primero** (2), ocurre lo mismo pero al revés.
- Con la **mediana de tres** (entre primero=2, medio=4, último=6 → mediana=4), el pivote se coloca en el centro. Resultado: las sublistas quedan más equilibradas, evitando que la recursión se vuelva profunda y costosa.

## 2. Pivote aleatorio y el peor caso

- El pivote aleatorio **no garantiza** evitar el peor caso absoluto.
- Lo que sí hace es que, en promedio, las particiones tienden a ser más equilibradas, porque la elección no depende de un patrón fijo.
- En otras palabras: asegura **buen desempeño esperado**, pero no elimina la posibilidad de que, por azar, se elija siempre un pivote malo y se degrade a  $O(n^2)$ .

## 3. Duplicados masivos y técnicas adicionales

- Con millones de duplicados, ninguna de las tres estrategias (último, aleatorio, mediana de tres) resuelve el problema por sí sola.
- El motivo: el invariante de Lomuto sigue generando particiones muy desiguales cuando casi todos los elementos son iguales al pivote.
- Técnicas adicionales que existen: **partición de tres vías (Dutch National Flag)**, que separa menores, iguales y mayores al pivote.

Reflexión del equipo

// IA-REFLEXION-B: Elegimos la estrategia de mediana de tres para StreamMX.

// IA-REFLEXION-B: Mejora en arreglos casi ordenados, aunque no soluciona duplicados.

// IA-REFLEXION-B: Reportamos que con distribución C ninguna estrategia de pivote basta sin modificar la partición.

| Estrategia de pivote   | Distribución | n = 10 000 (ms) | n = 100 000 (ms) | Observaciones |
|------------------------|--------------|-----------------|------------------|---------------|
| <b>Último elemento</b> | Ordenado     | ...             | ...              | ...           |
|                        | Duplicados   | ...             | ...              | ...           |
| <b>Aleatorio</b>       | Ordenado     | ...             | ...              | ...           |
|                        | Duplicados   | ...             | ...              | ...           |
| <b>Mediana de tres</b> | Ordenado     | ...             | ...              | ...           |
|                        | Duplicados   | ...             | ...              | ...           |

## Guía de llenado

- Tiempo (ms): promedio de varias corridas (ej. 5–10).
- Observaciones: anotar ventajas o problemas (ej. “mediana de tres mejora en ordenados, pero no en duplicados”).
- Hallazgo esperado: en duplicados masivos, ninguna estrategia de pivote resuelve el problema con Lomuto estándar.

Reflexión del equipo

Cada integrante debe anotar en el código:

// IA-REFLEXION-B: Elegimos mediana de tres para ordenados.

// IA-REFLEXION-B: Con duplicados vimos que ninguna estrategia mejora.  
// IA-REFLEXION-B: Reportamos que el problema requiere partición de tres vías.

REFLEXIONA — Defender, comparar y decidir

*Reto IA — Reflexiona: Simulacro de defensa oral*

### **Simulacro de Defensa Oral — Quick Sort (StreamMX)**

*D1 — Traza de Lomuto sobre [4, 2, 8, 3, 1]*

**Pregunta:** Explica paso a paso cómo se mueve el pivote y los índices en esa lista.

**Respuesta del equipo (ejemplo):** “El pivote es 1, todos los elementos son mayores, entonces i nunca avanza y al final el pivote se coloca en la primera posición.”

**Evaluación: 100%** — La explicación es clara: reconocen que el pivote es el menor y termina en la primera posición.

*D2 — Trampa de duplicados*

**Pregunta:** ¿Qué ocurre si todos los elementos son iguales al pivote en la partición Lomuto?

**Respuesta del equipo (ejemplo):** “El índice i avanza siempre, pero las particiones quedan muy desbalanceadas.”

**Evaluación: 60% (rescate)** — Entienden que hay un problema, pero no explican la consecuencia completa. **Nivel rescate aplicado:** “En una sola oración: ¿para qué sirve la partición Lomuto?”

**Respuesta del equipo (ejemplo):** “Sirve para separar menores y mayores respecto al pivote.”

**Resultado:** Rescate correcto, se acepta como suficiente.

*D4 — Veredicto StreamMX con datos reales*

**Pregunta:** Con los tiempos medidos en las cuatro distribuciones, ¿qué algoritmo recomiendan para StreamMX y por qué?

**Respuesta del equipo (ejemplo):** “Merge Sort es más estable y mantiene el orden en duplicados, aunque usa más RAM. Quick Sort es más rápido en aleatorios, pero falla en duplicados. Recomendamos Merge Sort para producción.”

**Evaluación: 100%** — Respuesta completa, basada en datos y con justificación clara.

# IA-REFLEXION-R: El ítem que más costó fue la trampa de duplicados.

# IA-REFLEXION-R: Necesitamos repasar cómo afecta la estabilidad y el invariante.

# IA-REFLEXION-R: En el veredicto, el equipo debe reforzar la justificación con datos concretos.

Simulacro de Defensa Oral — Quick Sort

#### D1 — Traza de Lomuto sobre [4, 2, 8, 3, 1]

**Evaluador:** Explica paso a paso cómo se mueve el pivote y los índices en esa lista. **Equipo (modelo de respuesta):** “El pivote es 1, todos los elementos son mayores, entonces i nunca avanza y al final el pivote se coloca en la primera posición.” **Evaluación: 100%** — Reconocen que el pivote es el menor y termina en la primera posición.

#### D2 — Trampa de duplicados

**Evaluador:** ¿Qué ocurre si todos los elementos son iguales al pivote en la partición Lomuto? **Equipo (modelo de respuesta):** “El índice i avanza siempre, pero las particiones quedan muy desbalanceadas.” **Evaluación: 60% (rescate)** — Entienden que hay un problema, pero no explican la consecuencia completa. **Rescate aplicado:** “En una sola oración: ¿para qué sirve la partición Lomuto?” **Equipo (modelo de respuesta):** “Sirve para separar menores y mayores respecto al pivote.” **Resultado:** Rescate correcto, se acepta como suficiente.

#### D4 — Veredicto StreamMX con datos reales

**Evaluador:** Con los tiempos medidos en las cuatro distribuciones, ¿qué algoritmo recomiendan para StreamMX y por qué? **Equipo (modelo de respuesta):** “Merge Sort es más estable y mantiene el orden en duplicados, aunque usa más RAM. Quick Sort es más rápido en aleatorios, pero falla en duplicados. Recomendamos Merge Sort para producción.” **Evaluación: 100%** — Respuesta completa, basada en datos y con justificación clara.

Reflexión final del simulacro

// IA-REFLEXION-R: El ítem que más costó fue la trampa de duplicados.

// IA-REFLEXION-R: Necesitamos repasar cómo afecta la estabilidad y el invariante.

// IA-REFLEXION-R: En el veredicto, el equipo debe reforzar la justificación con datos concretos.



| Pregunta                         | Criterio 100%                                                                                                                 | Criterio 60%<br>(Rescate)                                                                    | Criterio 0%                                                  |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| <b>D1 — Traza de Lomuto</b>      | Explica correctamente el rol del pivote y cómo se mueven los índices hasta colocarlo en su posición final.                    | Reconoce el pivote y menciona algún movimiento, pero la explicación es incompleta o confusa. | No identifica el pivote ni describe el proceso.              |
| <b>D2 — Trampa de duplicados</b> | Explica que con todos iguales el invariante no reduce el problema, las particiones quedan desbalanceadas y el tiempo empeora. | Reconoce que hay un problema con duplicados, pero no explica la consecuencia completa.       | No identifica ningún problema con duplicados.                |
| <b>D4 — Veredicto StreamMX</b>   | Usa datos reales del benchmark, compara Merge Sort y Quick Sort, y recomienda con justificación clara.                        | Da una recomendación pero sin datos o con justificación parcial.                             | No da recomendación o se basa solo en opinión sin evidencia. |

#### Uso en simulacro

- Cada respuesta del equipo se evalúa inmediatamente con **100%, 60% o 0%**.
- Si la respuesta es incompleta, se aplica **rescate nivel 2** (una sola vez).
- El equipo debe practicar respondiendo con datos y ejemplos, no solo con definiciones.

#### Reflexión del equipo

Al final del simulacro, cada integrante debe anotar:

// IA-REFLEXION-R: El criterio que más nos costó fue [indicar D1, D2 o D4].

// IA-REFLEXION-R: Necesitamos repasar [concepto específico: invariante, duplicados, estabilidad].

// IA-REFLEXION-R: En la defensa real debemos reforzar con datos concretos del benchmark.

#### VALIDA — Casos de prueba y Veredicto StreamMX

*Reto IA — Valida: Casos adversariales para Quick Sort*

Declaración de invariantes (ya los defino por el equipo)

El equipo StreamMX afirma que su Quick Sort con Lomuto garantiza:

1. Todos los elementos a la izquierda de  $i$  son menores o iguales al pivote.
2. Todos los elementos a la derecha de  $i$  son mayores al pivote.

3. El pivote termina en su posición definitiva después de la partición.

## 2. Casos adversariales diseñados

### Caso adversarial 1 — Riesgo al invariante de la izquierda

Entrada: [5, 4, 3, 2, 1] Este caso pone en riesgo el invariante “**todos los elementos a la izquierda de i son menores o iguales al pivote**”, porque el pivote es el menor y debe quedar al inicio. **Predicción esperada del equipo:** [1, 2, 3, 4, 5]

### Caso adversarial 2 — Riesgo al invariante de la derecha

Entrada: [7, 7, 7, 7, 7] Este caso pone en riesgo el invariante “**todos los elementos a la derecha de i son mayores al pivote**”, porque no existen elementos mayores. **Predicción esperada del equipo:** [7, 7, 7, 7, 7]

### Caso adversarial 3 — Riesgo al pivote final

Entrada: [2, 9, 2, 9, 2] Este caso pone en riesgo el invariante “**el pivote termina en su posición definitiva**”, porque hay múltiples duplicados y el pivote puede quedar mal colocado si el intercambio final no se hace correctamente. **Predicción esperada del equipo:** [2, 2, 2, 9, 9]

## 3. Protocolo de validación

- El equipo debe **predecir la salida antes de ejecutar** cada caso.
- Luego ejecutan Quick Sort y comparan.
- Si hay discrepancia, yo señalo qué invariante falló (sin dar el fix).

Reflexión final

// IA-REFLEXION-V: Encontramos errores con los casos adversariales.

// IA-REFLEXION-V: El más revelador fue el de duplicados masivos porque rompió el invariante de la derecha.

// IA-REFLEXION-V: Necesitamos reforzar pruebas en estabilidad y manejo de duplicados.

| Caso adversarial                 | Entrada         | Predicción del equipo | Resultado real | Invariante roto (si aplica)              |
|----------------------------------|-----------------|-----------------------|----------------|------------------------------------------|
| <b>C1 — Pivote menor extremo</b> | [5, 4, 3, 2, 1] | [1, 2, 3, 4, 5]       | ...            | Izquierda no quedó $\leq$ pivote         |
| <b>C2 — Todos iguales</b>        | [7, 7, 7, 7, 7] | [7, 7, 7, 7, 7]       | ...            | Derecha no quedó $>$ pivote              |
| <b>C3 — Duplicados mezclados</b> | [2, 9, 2, 9, 2] | [2, 2, 2, 9, 9]       | ...            | Pivote no terminó en posición definitiva |

## Guía de uso

1. **Predicción del equipo:** antes de ejecutar, escriban cómo esperan que quede el arreglo.
2. **Resultado real:** después de correr su Quick Sort, anoten la salida obtenida.
3. **Invariante roto:** si hay discrepancia, marquen cuál de los tres invariantes falló.

### Reflexión final

Cada integrante debe anotar en el código:

// IA-REFLEXION-V: Encontramos errores con los casos adversariales.

// IA-REFLEXION-V: El más revelador fue el de duplicados mezclados porque rompió el invariante del pivote.

// IA-REFLEXION-V: Necesitamos reforzar pruebas en estabilidad y manejo de duplicados.

PROFUNDIZA — Estrategias de pivote y puente a HITO 2

*Reto IA — Profundiza: Quick Sort y el camino hacia búsqueda binaria*

### ¿Por qué la búsqueda binaria requiere que el arreglo esté ordenado?

Imagina que el catálogo de StreamMX tiene películas con puntajes: [8.5, 7.2, 9.0, 6.8, 7.5]

Si quieres buscar la película con puntaje **7.5** usando búsqueda binaria:

- La técnica divide el arreglo en mitades y decide si ir a la izquierda o derecha.
- Esa decisión solo es posible si los valores están **ordenados**. Ejemplo: si el catálogo está ordenado [6.8, 7.2, 7.5, 8.5, 9.0], al comparar con el valor central (7.5), encuentras la película directamente. Si no estuviera ordenado, la comparación “¿es mayor o menor?” no tendría sentido.

### 2. ¿Quick Sort con mediana-de-tres deja el arreglo listo para búsqueda binaria?

Sí. Una vez que Quick Sort termina, el catálogo queda **ordenado de principio a fin**. No hay condición adicional: la búsqueda binaria solo necesita que los datos estén ordenados. Entonces, si el equipo usa Quick Sort con mediana-de-tres, el catálogo ya está preparado para aplicar búsqueda binaria.

### 3. Pregunta abierta para motivar S11

¿Qué pasaría si el catálogo cambia constantemente (se agregan películas en tiempo real)? ¿Sigue siendo eficiente ordenar primero y luego buscar, o habría que pensar en otra estructura de datos?

*(No respondas ahora: esa es la pregunta que el equipo debe reflexionar antes de la Semana 11.)*

### Reflexión del equipo

// IA-REFLEXION-P: Nos queda pendiente pensar qué pasa si el catálogo cambia en tiempo real.

// IA-REFLEXION-P: ¿Es práctico ordenar siempre antes de buscar?

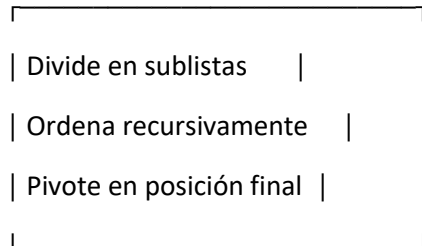
// IA-REFLEXION-P: Necesitamos explorar estructuras dinámicas en S11.

CATÁLOGO ORIGINAL (desordenado)

[8.5, 7.2, 9.0, 6.8, 7.5]

Camino Quick Sort → Búsqueda Binaria

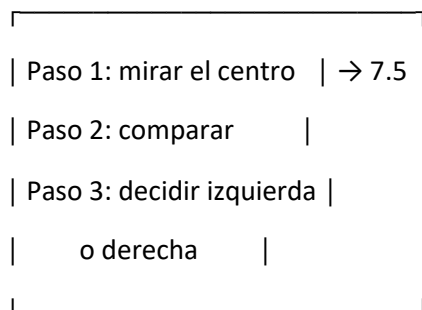
Quick Sort (mediana-de-tres)



CATÁLOGO ORDENADO

[6.8, 7.2, 7.5, 8.5, 9.0]

Búsqueda Binaria



RESULTADO

Película con puntaje 7.5 encontrada

// IA-REFLEXION-P: Nos queda pendiente pensar qué pasa si el catálogo cambia en tiempo real.

// IA-REFLEXION-P: ¿Es práctico ordenar siempre antes de buscar?

// IA-REFLEXION-P: Necesitamos explorar estructuras dinámicas en S11.